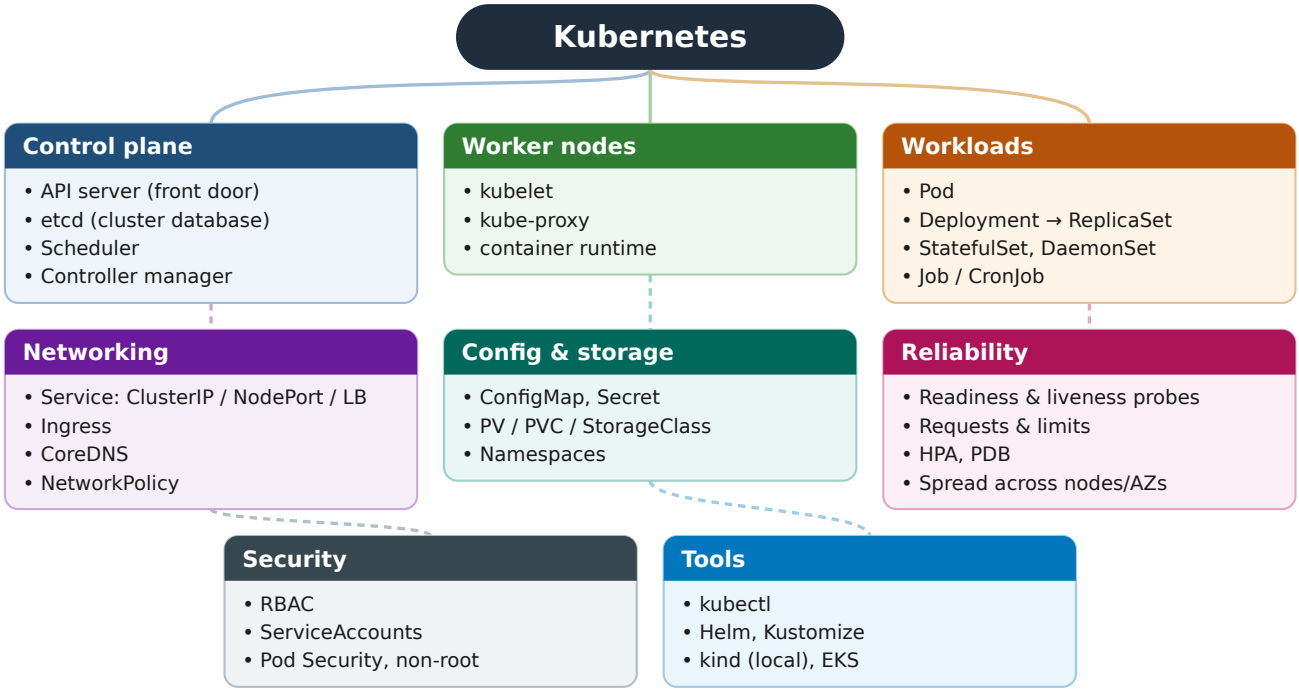
# 12. Kubernetes

DevOps, Cloud & Platform Engineering Notes · Section 12 of 18 · Thirupathi Keloth · v1.4

**In this section:** 12.1 Architecture · 12.2 Objects · 12.3 YAML basics · 12.4 Deployment + Service example · 12.5 ConfigMap, Secret, Ingress, PVC · 12.6 kubectl commands · 12.7 Scaling, RBAC, security · 12.8 Pod troubleshooting · 12.9 Helm · 12.10 Amazon EKS · 12.11 Scheduling and availability · 12.12 NetworkPolicy (firewall between pods) · 12.13 Kustomize (environment overlays without templates) · 12.14 More kubectl




## 12.1 Architecture

```
                CONTROL PLANE
┌──────────────┬──────────┬───────────┬────────────────────┐
│ API server   │   etcd   │ scheduler │ controller-manager │
└──────┬───────┴──────────┴───────────┴────────────────────┘
       │  (kubectl talks to the API server)
┌──────┴──────────────────┐   ┌─────────────────────────┐
│ WORKER NODE 1           │   │ WORKER NODE 2           │
│ kubelet, kube-proxy     │   │ kubelet, kube-proxy     │
│ container runtime       │   │ container runtime       │
│ [pod] [pod]             │   │ [pod] [pod]             │
└─────────────────────────┘   └─────────────────────────┘
```

| Component | Job |
|---|---|
| API server | Front door of the cluster. Everything goes through it |
| etcd | Key-value database storing the whole cluster state |
| Scheduler | Chooses a node for each new pod |
| Controller manager | Loops that make the actual state match the desired state |
| kubelet | Agent on each node that starts and watches pods |

| Component | Job |
|---|---|
| kube-proxy | Sets up Service networking on each node |
| Container runtime | Runs containers (containerd) |

**Core idea: desired state.** You declare what you want in YAML; Kubernetes keeps making it true (restarts, reschedules, scales).

## 12.2 Objects

| Object | Purpose |
|---|---|
| Pod | Smallest unit: one or more containers sharing network and storage. Temporary |
| ReplicaSet | Keeps N identical pods running |
| Deployment | Manages ReplicaSets: rolling updates and rollbacks. Use this for stateless apps |
| StatefulSet | Pods with stable names and storage (databases) |
| DaemonSet | One pod on every node (log/monitoring agents) |
| Job / CronJob | Run to completion / on a schedule |
| Service | Stable IP and DNS name in front of a set of pods |
| Ingress | HTTP(S) routing from outside to Services by host/path (needs an Ingress controller) |
| ConfigMap | Non-secret configuration |
| Secret | Sensitive data (base64-encoded, **not encrypted by default**: enable encryption, restrict access) |
| Namespace | Logical separation inside a cluster (dev, team-a) |
| PersistentVolume (PV) / Claim (PVC) | A piece of storage / a pod's request for storage |
| StorageClass | How storage is created dynamically (e.g. AWS EBS gp3) |
| ServiceAccount | Identity for pods |
| HPA | Horizontal Pod Autoscaler: changes replica count based on metrics |

## 12.3 YAML basics

```
key: value            # mapping
list:                 # list
  - item1
  - item2
nested:
  child: value        # indentation = structure (spaces only, never tabs)
```

Every Kubernetes object has: `apiVersion`, `kind`, `metadata` (name, labels), `spec` (desired state).

## 12.4 Deployment + Service example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 3                       # how many pods
  selector:
    matchLabels: { app: web }       # which pods this Deployment owns
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:                         # the pod blueprint
    metadata:
      labels: { app: web }          # must match the selector
    spec:
      containers:
        - name: web
          image: myuser/myapp:1.4.2 # pinned version, not latest
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef: { name: web-config }
            - secretRef:    { name: web-secrets }
          resources:
            requests: { cpu: 100m, memory: 128Mi }   # used for scheduling
            limits:   { cpu: 500m, memory: 256Mi }   # hard cap (over memory limit = OOMKilled)
          readinessProbe:           # ready to receive traffic?
            httpGet: { path: /health, port: 8000 }
            initialDelaySeconds: 5
          livenessProbe:            # still alive? (restart if not)
            httpGet: { path: /health, port: 8000 }
            initialDelaySeconds: 15
          securityContext:
            runAsNonRoot: true
            allowPrivilegeEscalation: false
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP                   # internal only
  selector: { app: web }            # send traffic to pods with this label
  ports:
    - port: 80                      # Service port
      targetPort: 8000              # container port
```

| Service type | Reachable from |
|---|---|
| ClusterIP (default) | Inside the cluster only. DNS: `web.<namespace>.svc.cluster.local` |
| NodePort | Each node's IP on a port 30000–32767 |
| LoadBalancer | A cloud load balancer (AWS NLB/ALB) |

**Probes:** readiness = should it get traffic; liveness = should it be restarted; startup = give slow apps time to start. **CPU units:** `1000m` = 1 core. **Memory:** `Mi`, `Gi`.

## 12.5 ConfigMap, Secret, Ingress, PVC

```
apiVersion: v1
kind: ConfigMap
metadata: { name: web-config }
data:
  APP_ENV: "prod"
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: web }
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: web, port: { number: 80 } }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources:
    requests: { storage: 10Gi }
```

Create secrets from the command line (not committed to Git): `kubectl create secret generic web-secrets --from-literal=DB_PASSWORD=...` or use External Secrets / Sealed Secrets.

## 12.6 kubectl commands

| Command | What it does |
|---|---|
| `kubectl get nodes` | List nodes |
| `kubectl get pods -A` / `-n dev` | Pods in all namespaces / one namespace |
| `kubectl get pods -o wide` | Extra info (node, IP) |
| `kubectl get all` | Common objects in the namespace |
| `kubectl apply -f file.yaml` | Create or update from YAML (declarative) |
| `kubectl delete -f file.yaml` | Delete what the file defines |
| `kubectl describe pod <name>` | Details and **Events** (first place to look) |
| `kubectl logs <pod>` / `-f` / `--previous` | Logs / follow / from the crashed container |
| `kubectl exec -it <pod> -- sh` | Shell inside a pod |
| `kubectl port-forward svc/web 8080:80` | Access a service from your laptop |
| `kubectl rollout status deploy/web` | Watch a rollout |
| `kubectl rollout history deploy/web` | Revision history |
| `kubectl rollout undo deploy/web` | Roll back |

| Command | What it does |
|---|---|
| `kubectl scale deploy/web --replicas=5` | Scale manually |
| `kubectl set image deploy/web web=myapp:1.5.0` | Update the image |
| `kubectl get events --sort-by=.lastTimestamp` | Recent cluster events |
| `kubectl top pods` / `top nodes` | CPU/memory use (needs metrics-server) |
| `kubectl config get-contexts` / `use-context` | Which cluster you're talking to |
| `kubectl explain deployment.spec` | Built-in documentation for fields |

Local clusters for practice: **kind** (`kind create cluster`), minikube, k3d.

## 12.7 Scaling, RBAC, security

**HPA:** `kubectl autoscale deploy web --cpu-percent=60 --min=2 --max=10` (requires metrics-server and resource requests).

**RBAC:**

| Object | Meaning |
|---|---|
| Role / ClusterRole | Allowed verbs (get, list, create...) on resources, in one namespace / cluster-wide |
| RoleBinding / ClusterRoleBinding | Gives a Role to a user, group or ServiceAccount |

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: read-pods, namespace: dev }
subjects:
  - kind: User
    name: alice
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Check permissions: `kubectl auth can-i delete pods -n dev --as alice`.

**Security checklist:** RBAC least privilege, non-root containers, Pod Security Standards (`restricted`), NetworkPolicies (default deny, then allow what's needed), resource limits, scanned and pinned images, secrets encrypted and access-controlled, no `privileged: true`.

## 12.8 Pod troubleshooting

| Status | Usual cause | First command |
|---|---|---|
| Pending | Not enough CPU/memory, node selector/taints, PVC not bound | `kubectl describe pod` → Events |
| ImagePullBackOff / ErrImagePull | Wrong image name/tag, private registry without credentials | `kubectl describe pod` |
| CrashLoopBackOff | App starts and crashes: bad config, missing env/secret, wrong command | `kubectl logs <pod> --previous` |
| OOMKilled | Memory limit too low or a memory leak | `kubectl describe pod` (Last State) |
| Running but not Ready | Readiness probe failing | `kubectl describe pod`, check the probe path/port |
| Service not reachable | Selector labels don't match pods → no endpoints; wrong targetPort | `kubectl get endpoints <svc>` |
| Ingress 404/502 | Wrong host/path, wrong service name/port, no Ingress controller | `kubectl describe ingress` |

**Method:** `get` → `describe` (Events) → `logs` (and `--previous`) → `exec` inside to test → check Service endpoints → check node resources.

## 12.9 Helm

| Term | Meaning |
|---|---|
| Helm | Package manager for Kubernetes |
| Chart | A package of templated Kubernetes YAML |
| `values.yaml` | Default settings; override per environment |
| Release | One installed instance of a chart |
| Repository | Where charts are published |

```
mychart/
├── Chart.yaml          # name, version
├── values.yaml         # replicaCount: 2, image.tag: "1.4.2"
└── templates/
    ├── deployment.yaml # uses {{ .Values.replicaCount }}, {{ .Values.image.tag }}
    └── service.yaml
```

| Command | What it does |
|---|---|
| `helm create mychart` | Create a chart skeleton |
| `helm repo add bitnami https://charts.bitnami.com/bitnami` / `helm repo update` | Add / refresh a repo |
| `helm install web ./mychart -f values-prod.yaml -n prod` | Install a release |
| `helm upgrade --install web ./mychart --set image.tag=1.5.0` | Upgrade (or install) |
| `helm list -A` | List releases |

| Command | What it does |
|---|---|
| `helm history web` / `helm rollback web 2` | History / roll back to revision 2 |
| `helm template ./mychart` | Render the YAML locally without installing |
| `helm lint ./mychart` | Check the chart |
| `helm uninstall web` | Remove the release |

## 12.10 Amazon EKS

| Term | Meaning |
|---|---|
| EKS | Managed Kubernetes control plane by AWS (billed per hour) |
| Managed node group | EC2 worker nodes managed by EKS |
| Fargate profile | Run pods without managing nodes |
| VPC CNI | Pods get real VPC IP addresses |
| AWS Load Balancer Controller | Creates ALBs/NLBs for Ingress and Services |
| IRSA / EKS Pod Identity | Give pods an IAM role, without keys |
| Access entries | Map IAM users/roles to Kubernetes permissions |

| Command | What it does |
|---|---|
| `aws eks update-kubeconfig --name my-cluster --region ap-south-1` | Connect kubectl to the cluster |
| `eksctl create cluster --name lab --nodes 2` | Quick cluster for labs |
| `eksctl delete cluster --name lab` | **Delete after the lab** |

EKS costs money every hour (control plane + nodes + load balancers + NAT). Build with Terraform, use it, and destroy it the same day in labs.

## 12.11 Scheduling and availability

| Feature | Purpose |
|---|---|
| nodeSelector / node affinity | Run pods only on certain nodes (e.g. GPU nodes) |
| Pod anti-affinity / topologySpreadConstraints | Spread replicas across nodes/AZs so one failure doesn't take all pods |
| Taints and tolerations | A tainted node repels pods unless they tolerate the taint (dedicated nodes) |
| PodDisruptionBudget (PDB) | Keeps a minimum number of pods running during node drains/upgrades |
| Init containers | Run setup steps before the main container starts |
| Sidecar containers | Helper container in the same pod (log shipper, proxy) |
| `kubectl drain <node>` / `cordon` / `uncordon` | Safely empty a node for maintenance / stop new pods / allow again |

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: web-pdb }
spec:
  minAvailable: 2
  selector:
    matchLabels: { app: web }
```

## 12.12 NetworkPolicy (firewall between pods)

```
# 1) Deny all incoming traffic to pods in the namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-ingress, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Ingress]
---
# 2) Allow only the web pods to reach the database on 5432
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-web-to-db, namespace: prod }
spec:
  podSelector:
    matchLabels: { app: db }
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: web }
      ports:
        - protocol: TCP
          port: 5432
```

NetworkPolicies only work if the cluster's CNI supports them (Calico, Cilium; on EKS enable network policy in the VPC CNI).

## 12.13 Kustomize (environment overlays without templates)

```
k8s/
├── base/            (deployment.yaml, service.yaml, kustomization.yaml)
└── overlays/
    ├── dev/         kustomization.yaml → replicas: 1, image tag dev
    └── prod/        kustomization.yaml → replicas: 3, image tag 1.4.2
```

Apply with `kubectl apply -k k8s/overlays/prod`. **Helm vs Kustomize:** Helm = templating + packaging + releases; Kustomize = patching plain YAML. Both work with Argo CD.

## 12.14 More kubectl

| Command | Purpose |
|---|---|
| `kubectl get deploy web -o yaml` | Full live object as YAML |
| `kubectl diff -f file.yaml` | What would change before applying |
| `kubectl create deployment web --image=nginx --dry-run=client -o yaml > web.yaml` | Generate starter YAML |

| Command | Purpose |
|---|---|
| `kubectl config set-context --current --namespace=dev` | Change default namespace |
| `kubectl run tmp --rm -it --image=busybox -- sh` | Temporary debug pod (test DNS: `nslookup web`) |
| `kubectl debug -it <pod> --image=busybox` | Attach a debug container to a running pod |
| `kubectl get pods -l app=web` | Filter by label |
| `kubectl api-resources` | All resource types in the cluster |

**REMEMBER**

- Debug order: **get → describe (Events) → logs --previous → exec → get endpoints**.
- **Labels connect everything**: Deployment selector ↔ pod labels ↔ Service selector.
- **Readiness** = send traffic? **Liveness** = restart? **Requests** = scheduling, **limits** = hard cap.